Semantic Authority, Language Bundles, and MM2-Centered Evaluation

A Conceptual Primer for Verified Rewritable Language Engines

March 23, 2026

Abstract

This note distills a coherent architecture for verified language engineering across four linked layers: language specification, semantic packaging, execution contracts, and backend realization. Its central thesis is that the meaning of a language should be exposed once, explicitly, in a machine-usable semantic authority that both proofs and runtimes consume. The note develops five ideas. First, a language should be authored as a neutral *language bundle* rather than as a low-level record tied to one proof assistant or one runtime. Second, surface syntax, statement classification, and desugaring are part of the language and must not be silently reconstructed in a backend. Third, graph-structured language theories (GSLTs), operational-semantic logical frameworks (OSLFs), and native-type extraction (NTT) should be treated as derived semantic views of the same authority rather than competing sources of truth. Fourth, runtime architecture should follow the rule *profiles choose, contracts execute*: backend decisions come from certified summaries, while detailed contracts drive actual execution. Fifth, recursive evaluation over rewritable languages is best approached through an MM2-centered explicit-state protocol with request/result/join facts, memoization, and persistent rules, while keeping host code as a supervisory shell and benchmark harness. The note is written as a primer for future work and future chats: it records the conceptual decisions, anti-patterns, proof strategy, and migration plan that emerged from a long design iteration.

Contents

1 Purpose and Scope **1**

2 The Core Architectural Shift **2**
- 2.1 From split authority to semantic authority . . . 2
- 2.2 What counts as "part of the language" . . . 2

3 Language Bundles Rather Than Low-Level Records **3**
- 3.1 Why low-level `LanguageDef` authoring failed . . . 3
- 3.2 The neutral bundle schema . . . 3
- 3.3 A three-layer internal structure . . . 4
- 3.4 Why this should remain close to runtime DSLs . . . 4

4 Surface Syntax Is Part of the Language **4**
- 4.1 The parser fallacy . . . 4
- 4.2 Surface elaboration by language rule . . . 5
- 4.3 Token classes and the RawToken issue . . . 5

4.4 Shared syntax families versus dialect overlays . . . 5

5 GSLT, OSLF, and NTT as Derived Semantic Views **5**
5.1 The semantic pipeline . . . 5
5.2 What NTT is good for . . . 6
5.3 Profiles choose, contracts execute . . . 6

6 Premises, Derived Relations, and Effects **6**

7 Backend-Neutral Execution Architecture **7**
7.1 Why backend-neutrality matters . . . 7
7.2 Prepared programs and runtime bundles . . . 7
7.3 The anti-pattern of backend-shaped semantics . . . 8

8 MM2-Centered Evaluation Rather Than One-Shot Compilation **8**
8.1 What failed . . . 8
8.2 The right target: explicit-state evaluator protocol . . . 8
8.3 Head-specialized protocol facts . . . 9
8.4 Persistent rules versus copy budgets . . . 9
8.5 Memoization as protocol, not host heuristic . . . 9

9 Proof Architecture for Recursive Evaluation **10**
9.1 Fuel is an algorithmic device, not the final semantics . . . 10
9.2 Three-layer proof picture . . . 10
9.3 Ideal protocol first, concrete engine second . . . 10
9.4 Linearity claims for memoized recursion . . . 11

10 A Generic Migration Path **11**
10.1 Phase 1: Author the bundle . . . 11
10.2 Phase 2: Derive proof views . . . 11
10.3 Phase 3: Derive runtime artifacts . . . 11
10.4 Phase 4: Realize on one or more backends . . . 11
10.5 Phase 5: Benchmark and refine . . . 11

11 Anti-Patterns and Design Rules **12**
11.1 Anti-patterns . . . 12
11.2 Positive design rules . . . 12

12 Open Problems and Research Directions **12**

13 Primer Checklist for Future Work and Future Chats **13**

14 Conclusion **13**

1 Purpose and Scope

The goal of this paper is to state, as cleanly as possible, the architectural lessons of a long design process around verified language runtimes. The intended setting is a family of languages with rich

rewriting behavior, possible surface syntax variation, theorem-proving or symbolic sublanguages, and multiple execution backends. The main design tension is familiar:

> How can one keep semantics authoritative, proofs reusable, and runtimes replaceable, while still allowing aggressive engineering and performance work?

The answer developed here is deliberately conceptual. It is not a tutorial for one particular repository, and it is not an implementation manual for one particular runtime. Instead it offers a stable blueprint.

Primary claim. The meaning of a language must be exposed once, explicitly, in a neutral semantic bundle. Every other layer—surface syntax, proof packaging, native-type extraction, runtime contracts, backend selection, evaluator protocols, and optimization strategies—should be derived from, checked against, or constrained by that bundle.

Secondary claim. A large amount of what is often treated as parser logic, runtime heuristics, or backend convenience is actually part of the language. In particular, statement classification, desugaring, scoping discipline, effect signatures, and execution-lane selection should not be silently reinvented by a backend.

Practical claim. For recursive evaluation and symbolic control flow, an MM2-centered explicit-state protocol is a better long-term target than one-shot compilation of each encountered term. Host code remains useful, but primarily as loader, supervisor, watchdog, benchmark harness, and fallback oracle.

2 The Core Architectural Shift

2.1 From split authority to semantic authority

The most important mistake to avoid is *split authority*. Split authority occurs when different parts of the system silently decide different parts of the language:

- a parser decides which forms are rules and which are facts;
- a runtime decides which operations are effectful and which are pure;
- a proof layer decides a clean rewrite theory, while a backend reconstructs a nearby but different operational classification;
- one artifact claims to represent the language, but critical semantics are encoded in side files, callbacks, or ad hoc dispatch logic.

The corrective principle is simple:

> **Semantic authority principle.** Every semantically load-bearing distinction must appear explicitly in the authoritative language bundle, or be derivable from it by a checked elaboration.

This principle has two immediate corollaries.

1. Runtimes may *realize* semantics, but they may not *invent* them.
2. Proof layers should verify a well-designed semantic bundle, not force authors to encode languages as giant low-level records.

2.2 What counts as "part of the language"

The following are all part of the language and should therefore appear in the bundle:

- token classes and surface grammar,
- statement forms and top-level classification,
- desugaring and elaboration into kernel terms,
- kernel constructors and equations,
- rewrite rules and premise relations,
- scope and binding structure,
- effect signatures and contract classes,
- execution-lane eligibility and residual policy,
- surface-level variants and semantics variants.

The following should *not* silently appear only in a backend:

- "the last form is the query,"
- "a form headed by = is a rewrite rule,"
- "this builtin is memo-safe,"
- "this branch may mutate space,"
- "this operation must fail closed rather than fall back symbolically."

3 Language Bundles Rather Than Low-Level Records

3.1 Why low-level `LanguageDef` authoring failed

A low-level record can be useful as an internal kernel representation, but it is usually a bad authoring interface. Typical failure modes are:

- it is verbose and discourages direct reading;
- it hides structure by flattening everything into constructor lists;
- it pushes semantics into side comments or naming conventions;
- it does not align with how runtime authors think about a language;
- it creates pressure to add ad hoc escape hatches.

So the fix is not to abandon semantic authoring in Lean or another proof assistant. The fix is to replace the low-level authored object.

3.2 The neutral bundle schema

The proposed authored object is a *language bundle*. Conceptually:

```
LanguageBundle = {
  name,
  options,
  tokens,
  surface,
  kernel,
  equations,
  rewrites,
  derived_relations,
  effects,
  scope_specs,
  metadata
}
```

The bundle is deliberately neutral:

- it is not tied to a specific Rust macro implementation;
- it is not tied to one proof assistant's internal inductive encoding;
- it is expressive enough to derive proof-facing and runtime-facing views.

3.3 A three-layer internal structure

A useful refinement is to split the bundle into three internal layers.

Surface layer. Tokens, notation, precedence, sugar, top-level statement forms, elaboration rules.

Kernel layer. Typed constructors, equations, rewrites, explicit premise families, explicit scoping structure.

Operational layer. Effects, resource classes, eligibility, residual policy, runtime hints, protocol motifs.

This leads to a healthy derivation picture:

$$\text{LanguageBundle} \longrightarrow \text{SurfaceDef},\ \text{KernelDef},\ \text{RuntimeContracts},\ \text{Profiles},\ \text{ProtocolSpecs}.$$

3.4 Why this should remain close to runtime DSLs

Human-facing authoring syntax should stay close to practical runtime DSLs when possible. If runtime authors naturally think in a declarative macro language, that style should be preserved. The lesson is not that proof-side authoring must look alien. The lesson is that the authored object should elaborate into a principled bundle rather than into runtime-specific ad hoc code.

4 Surface Syntax Is Part of the Language

4.1 The parser fallacy

A recurring error in language engineering is to treat surface syntax as "mere parsing" and therefore as implementation detail. The deeper lesson is:

> Surface syntax is not only concrete syntax. It includes classification, elaboration, and policy.

This means at least four distinct layers must be separated.

1. **Raw reader.** Bytes or characters to tokens or grouped structure.
2. **Concrete syntax.** Parse tree or syntax tree.
3. **Surface classification.** Rule, fact, query, import, directive, test, mutation, and so on.
4. **Elaboration.** Surface objects to kernel terms and statements.

The first layer is often trusted infrastructure. The rest should be language authority.

4.2 Surface elaboration by language rule

In some language families, especially symbolic ones, large parts of surface syntax can be represented as elaboration rules or surface rewrites in the language itself. This is valuable. It shrinks the trusted frontend boundary and aligns surface semantics with the rest of the language.

But this does *not* imply that parsing disappears. What it implies is:

> The trusted reader should be made as small as possible, and everything above it should be made explicit and language-governed.

4.3 Token classes and the RawToken issue

Raw-token-heavy languages, theorem-prover languages, and parser-light syntaxes stress the bundle design. The right response is not a one-off hack. It is to make token classes first-class. For example:

```
tokens {
  ident Label;
  raw   Sym;
  raw   ProofTok;
  exact "$c";
  exact "$p";
  exact "$=";
  exact "$.";
}
```

This permits languages such as Metamath-like systems to remain principled even when their surface shape is highly token-driven.

4.4 Shared syntax families versus dialect overlays

A practical architecture distinguishes:

- a *syntax-family bundle* describing a shared concrete syntax, and
- a *dialect overlay* describing classification, elaboration, and program policy.

This is appropriate only when the concrete syntax is genuinely shared. Otherwise one needs separate syntax-family bundles or explicit grammar overlays.

5 GSLT, OSLF, and NTT as Derived Semantic Views

5.1 The semantic pipeline

The conceptual pipeline developed here is:

$$\text{LanguageBundle} \longrightarrow \text{KernelDef} \longrightarrow \text{GSLT} \longrightarrow \text{OSLF} \longrightarrow \text{NTT/Profile}.$$

Each arrow is a derivation, not a change of source of truth.

GSLT layer. A graph-structured language theory packages the language as a structured semantic object, potentially indexed over a family of variants or vertices.

OSLF layer. An operational-semantic logical framework packages reduction, modalities, Galois structure, and semantic operators.

NTT layer. Native-type extraction derives execution-relevant classifications such as determinism, effectfulness, resource class, rule family, premise usage, and lane eligibility.

5.2 What NTT is good for

NTT is most valuable when it tells the runtime facts the runtime should not reconstruct heuristically:

- rewrite vs query vs control vs effect,
- ordinary forward vs premise-aware vs compat-head vs symbolic-output,
- pure vs effectful,
- deterministic vs branching,
- memo-safe vs non-memo-safe,
- scope kind,
- boundary/witness/residual policy.

NTT is less useful if treated as magical optimization dust. Its role is to expose semantic classification to the implementation layer.

5.3 Profiles choose, contracts execute

A concise rule for runtime architecture is:

> **Profiles choose, contracts execute.**

A runtime profile or native-type summary should be used to decide:

- which execution lane is semantically licensed,
- whether memoization is allowed,
- whether residual fallback is legal,
- which fragment family is active.

Detailed execution artifacts then drive actual execution. This preserves semantic authority while still allowing engineered backends.

6 Premises, Derived Relations, and Effects

One of the deepest lessons is that a language bundle should not contain semantically opaque premise names. Every premise occurrence should fall into one of three classes.

Structural premise. Discharged internally by matching, equations, or rewrites.

Derived relation. Defined by internal relation rules in the bundle and therefore semantically visible.

Effectful premise. Declared as an effect with explicit signature and explicit contract class.

Anything else creates semantic drift. In practice, this means that operations like store access, space mutation, compatibility checks, external calls, or grounded host services must either be internalized as derived relations or explicitly marked as effects.

7 Backend-Neutral Execution Architecture

7.1 Why backend-neutrality matters

A runtime implementation may change dramatically over time: interpreter, graph engine, path map, bytecode VM, JIT, symbolic evaluator, or proof-producing engine. The enduring value lies in the semantic boundary, not in any single engine.

Therefore the right design principle is:

> **A runtime backend may be replaced without changing the language bundle.**

This requires a backend-neutral execution interface.

7.2 Prepared programs and runtime bundles

The practical shape is:

- a prepared program or command IR,
- a runtime bundle containing manifest, contracts, profile, and protocol spec,
- a supervisory host shell,
- one or more execution engines.

The host shell should do only those things that are not themselves language meaning:

- loading and integrity checking,
- launching evaluation,
- collecting results,
- enforcing external resource limits,
- benchmarking and cross-backend comparison.

7.3 The anti-pattern of backend-shaped semantics

A recurring anti-pattern is to let the semantics collapse into whatever the current backend happens to prefer. This leads to:

- frontends shaped by current parser shortcuts,
- profiles shaped by current graph-engine quirks,
- memo policies shaped by current cache internals,
- proofs shaped by current runtime implementation rather than language meaning.

This anti-pattern must be resisted from the start.

8 MM2-Centered Evaluation Rather Than One-Shot Compilation

8.1 What failed

A key engineering insight is that compiling each encountered term as a one-shot job is too coarse for recursive rewriting and symbolic control. Typical failure modes include:

- recursive user calls nested inside intrinsic calls,
- branch/control constructs such as `if` and `case`,
- sequential destructuring such as `let*`,
- multi-step logic-style chaining,
- repeated recompilation of large partially reduced terms.

The correct lesson is not that an engine like MM2 lacks expressive power. The lesson is that it must be used at the right granularity.

8.2 The right target: explicit-state evaluator protocol

The preferred long-term target is an MM2-centered explicit-state protocol. Conceptually:

- requests encode pending computation,
- results encode completed subcomputations,
- wait/join facts encode dependencies,
- memo facts encode reusable pure subresults,
- rule persistence avoids repeated rule installation.

Typical fact families are:

```
req_call_fib sid n
req_if sid cond then else
req_mul sid a b
wait_if sid then else
wait_mul_l sid rhs
wait_mul_r sid lhsValue
res sid value
memo key value
inflight key owner
wait_memo key waiter
```

This is superior to a host-style call stack model because it aligns directly with graph-like and rewriting engines.

8.3 Head-specialized protocol facts

Index-sensitive backends benefit greatly from head-specialized facts. Rather than generic requests carrying nested payloads, one should push the operator or constructor into the fact head itself. This improves:

- index selectivity,
- rule discrimination,
- protocol readability,
- performance profiling,
- backend portability.

8.4 Persistent rules versus copy budgets

A particularly important insight is that single-use execution rules are semantically awkward for recursive evaluation protocols. If an engine permits a persistent rule class, that should be preferred. Then:

- protocol rules are persistent,
- request/result/wait/memo facts are ephemeral,

- semantic models do not depend on arbitrary copy budgets.

If an engine only offers single-use rules, bounded copies can still be used as an implementation refinement, but they should not be treated as the semantic model.

8.5 Memoization as protocol, not host heuristic

Naive recursion such as Fibonacci is not made efficient merely by using a graph engine. The right optimization is protocol-level memoization. For pure deterministic recursive calls:

1. check a memo table encoded as facts,
2. share in-flight subproblems,
3. publish results once,
4. wake all waiters.

This keeps memoization explicit, verifiable, and portable across engines.

9 Proof Architecture for Recursive Evaluation

9.1 Fuel is an algorithmic device, not the final semantics

A crucial proof-level lesson is that a memoized evaluator should usually *not* be proved correct by direct equality with a fuel-bounded non-memoized function at the same fuel. The reason is simple: memoization can legitimately reuse work computed in a larger-fuel context. Therefore the correct semantic target is a fuel-free semantic relation.

9.2 Three-layer proof picture

The durable proof architecture has three layers.

Fuel-free semantics. An inductive big-step or small-step semantic relation giving the intended meaning.

Executable evaluators. Functions such as `eval` and `evalMemo` used for execution and testing.

Trace objects. Proof-relevant traces recording memo hits, misses, protocol steps, and cost.

The right theorem pattern is then:

- evaluator soundness with respect to semantics,
- completeness above a sufficient fuel threshold,
- trace soundness and preservation of semantic invariants,
- simulation of traces into protocol traces.

9.3 Ideal protocol first, concrete engine second

Another important lesson is to prove protocol semantics before engine semantics.

First prove:

> Trace semantics simulate an ideal persistent-rule request/result/join protocol.

Then prove:

> The concrete engine faithfully realizes that protocol under the relevant scheduling or copy-budget assumptions.

This separation makes the semantic work last even when the backend evolves.

9.4 Linearity claims for memoized recursion

When proving complexity properties such as linearity for memoized Fibonacci, it is wise to separate:

1. semantic uniqueness of subproblems,
2. protocol-level counts such as miss count or distinct memo entries,
3. concrete engine runtime behavior under a chosen indexing model.

This avoids overclaiming wall-clock performance while still proving useful asymptotic behavior.

10 A Generic Migration Path

The architecture proposed here supports a clear migration path.

10.1 Phase 1: Author the bundle

Design the language in the neutral bundle syntax. Keep surface, kernel, derived, and effectful parts explicit.

10.2 Phase 2: Derive proof views

Derive:

- kernel semantic views for GSLT/OSLF,
- native-type or profile views,
- protocol motifs and effect signatures.

10.3 Phase 3: Derive runtime artifacts

Export manifests, profiles, contracts, prepared IR, protocol specs, and grounded operation registries.

10.4 Phase 4: Realize on one or more backends

Provide one or more execution engines:

- direct host evaluator,
- graph/rewrite engine,
- bytecode VM,
- JIT,
- theorem-proving or proof-producing evaluator.

These remain replaceable because the semantic authority is stable.

10.5 Phase 5: Benchmark and refine

Once multiple realizations exist, benchmark them against the same semantic contracts and the same protocol counters. This is essential: performance work should become informative, not semantics-defining.

11 Anti-Patterns and Design Rules

11.1 Anti-patterns

1. **Parser authority in the backend.** Classifying statements in backend code without language authority.
2. **Hidden premise semantics.** Premise names whose meaning lives only in a runtime.
3. **Backend-shaped semantics.** Treating current engine convenience as language meaning.
4. **Ad hoc evaluator loops.** Repeatedly recompiling whole terms rather than compiling evaluator state.
5. **Copy-budget semantics.** Treating operational budgets as if they were semantic rules.
6. **Monolithic low-level authoring.** Forcing humans to author giant kernel records directly.
7. **Deceptive demos.** Polishing user-facing demonstrations before the authority boundary is correct.

11.2 Positive design rules

1. Author a neutral bundle, not backend glue.
2. Keep surface, kernel, and operational layers explicit.
3. Make token classes first-class.
4. Make every premise structural, derived, or effectful.
5. Derive GSLT/OSLF/NTT views from the same bundle.

6. Let profiles choose and contracts execute.
7. Treat host code as supervisory shell unless semantics truly requires more.
8. Compile recursive evaluation into explicit request/result/join protocols.
9. Prove semantics against ideal protocol models before backend details.
10. Benchmark multiple realizations against one semantic authority.

12 Open Problems and Research Directions

Several deep questions remain open and are worth turning into a research program.

Completeness for all reasonable GSLTs. What class of GSLTs is represented by the proposed bundle schema, and what extensions are needed for parser-light, raw-token, or surface-rewrite-heavy languages?

Binding and scope. Can scope, binding, and capture-avoiding substitution be fully first-class in the bundle without exploding complexity?

Protocol extraction. Can evaluator protocols for recursive fragments be extracted automatically from bundle-level declarations and fragment certifications?

Backend refinement theorems. How can one systematically refine ideal persistent-rule protocols to concrete graph or path-map engines with engine-specific scheduling and storage disciplines?

Complexity accounting. Which complexity measures are stable enough across backends to be worthwhile proof targets?

Round-trip laws. Can one establish robust round-trip laws among authoring DSLs, neutral bundles, proof views, and runtime artifacts?

13 Primer Checklist for Future Work and Future Chats

The following checklist is intended as a compact operational memory.

Question	Preferred answer
What is the source of truth?	A neutral language bundle, not a backend DSL and not a low-level record.
Is parsing "just parsing"?	No. Classification, elaboration, and policy are part of the language.
What should GSLT/OSLF/NTT operate on?	A kernel view derived from the language bundle.
How should runtime decisions be made?	By certified profiles and contracts, not heuristics.

How should recursion be evaluated?	By an explicit-state protocol with request/result/join facts and memoization.
Should the host own semantics?	No. The host is loader, supervisor, benchmark harness, and fallback shell.
What if a backend evolves?	Keep the bundle and protocol stable; treat backend behavior as a refinement.
How should raw-token-heavy languages be handled?	Make token classes first-class in the bundle.
How should effects be represented?	Explicitly, with signatures and contract classes.
What is the biggest anti-pattern?	Split authority: semantics silently divided across frontend, proof layer, and backend.

14 Conclusion

A durable architecture for rewritable language engines emerges once one stops asking whether the proof layer or the runtime should be "the" language definition. Neither should be. The correct primary object is a neutral semantic bundle that is pleasant enough to author, precise enough to verify, rich enough to derive semantic and operational views, and explicit enough to constrain multiple realizations. From that point of view, GSLT, OSLF, NTT, surface elaboration, runtime profiles, contracts, and MM2-style evaluator protocols are not rival formalisms. They are coordinated projections of a single language authority.

If this architecture is respected, then practical runtime innovation becomes less dangerous. One can experiment with graph engines, path maps, bytecode, JIT compilation, protocol memoization, or persistent rule extensions without forcing every optimization back into the semantics. Conversely, one can improve semantic clarity—for example by separating surface and kernel language layers, or by turning hidden services into explicit effects—without rewriting every backend from scratch.

That is the main conceptual result: semantic authority should be explicit, backend-neutral, and derivationally rich enough that proofs and runtimes can both be aggressive without drifting apart.